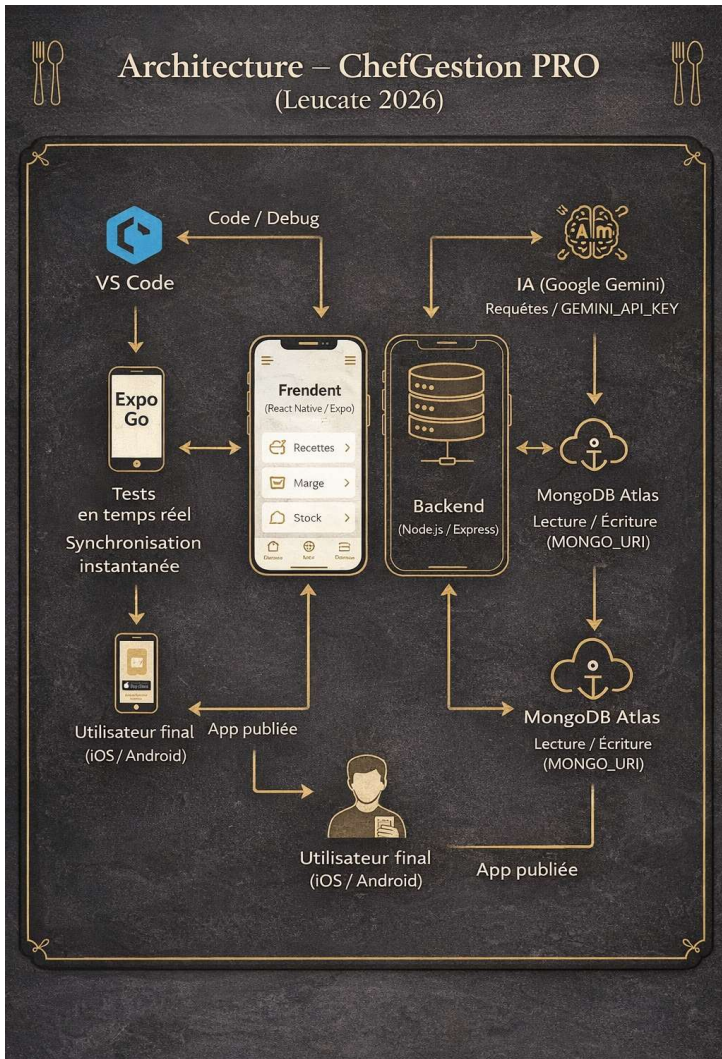




**Note du Chef :** Cet outil est le centre de commandement de la *Cabana del tío*. Il fusionne l'exigence du métier de cuisinier avec la puissance de l'IA pour automatiser le HACCP, le scan de factures et la surveillance des ratios. Ce n'est pas juste du code, c'est l'organisation de ma cuisine mise en poche.

## Schéma d'architecture de l'application ChefGestion PRO :



---

## Sommaire

- 1.  Fonctionnalités
- 2.  Lancement Rapide (Workflow)
- 3.  Architecture Technique Détaillée
- 4.  Écosystème Cloud & API
- 5.  Procédure de Sauvegarde & Déploiement (Git)
- 6.  Distribution (APK & iOS)
- 7.  Outils & Services Utilisés
- 8.  Limitations des API (Versions Gratuites)
- 9.  Dépannage (Troubleshooting)

---

## Fonctionnalités

| Module                                                                                                 | Description                                                                                                   |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
|  <b>Dashboard</b>   | KPIs en temps réel et alertes basés sur tes stocks.                                                           |
|  <b>Scanner OCR</b> | Analyse de températures frigos, factures et étiquettes sanitaires via <b>Gemini Vision AI</b> et expo-camera. |
|  <b>Ratios</b>      | Analyse de performance avec graphiques animés (reanimated).                                                   |
|  <b>Outils</b>      | Calculateur de marge et export PDF des fiches techniques (expo-print).                                        |
|  <b>HACCP</b>       | Relevés de températures et archivage numérique des étiquettes sanitaires.                                     |
|  <b>Paramètres</b>  | Configuration des clés API et gestion de ton profil.                                                          |

## Installation et Lancement Rapide (Workflow)

## Installation des dépendances

Terminal `# Se placer dans le dossier /frontend cd frontend`

`npm install`

## Démarrage du Frontend (Mobile)

Ton script est configuré pour ton IP fixe (192.168.1.122) afin de garantir la liaison Wi-Fi avec ton smartphone.

Terminal `# Se placer dans le dossier /frontend cd frontend`

`npm run start`

**Action :** Lance Metro, vide le cache (-c) et expose l'app sur ton réseau local.

**Si cela ne marche pas,** utilise cette commande spécifique pour forcer Metro à diffuser sur ton IP locale et éviter l'erreur de téléchargement :

Terminal

`$env:REACT_NATIVE_PACKAGER_HOSTNAME="192.168.1.122"; npx expo start --lan -c`

- **\$env:**... : Indique à ton téléphone où se trouve ton PC.
- **--lan** : Force la connexion locale.
- **-c** : Vide le cache pour partir sur une base propre.

## Démarrage du Backend (Serveur)

Terminal `# Se placer dans le dossier /backend cd backend`

`npm run dev`

**Action :** Lance le serveur Express sur le port local (généralement 5001) pour tester tes API avant de "pusher" sur Render.

---

## Architecture Technique Détaillée

Voici la cartographie complète de ton application pour une maintenance sans erreur.

## FRONTEND (React Native / Expo 54)

|                         |                                               |
|-------------------------|-----------------------------------------------|
| frontend/               |                                               |
| ├── app/                | # 🕒 Navigation (Expo Router)                  |
| │   ├── _layout.tsx     | # Root : Fonts (Cinzel), AuthProvider, Splash |
| │   ├── (auth)/         | # Connexion & Inscription                     |
| │   └── (tabs)/         | # Navigation principale (Bottom Tabs)         |
| │       ├── index.tsx   | # Dashboard (Accueil)                         |
| │       ├── scanner.tsx | # Interface Caméra + Analyse IA               |
| │       ├── ratios.tsx  | # Graphiques et statistiques                  |
| │       ├── tools.tsx   | # Calculateurs de cuisine                     |
| │       ├── haccp.tsx   | # Tableaux de relevés sanitaires              |
| │       ├── more.tsx    | # Paramètres, Fournisseurs, Profil            |
| │       └── _layout.tsx | # Design Noir & Or (Icons & Labels centrés)   |
| ├── components/         | # 🧩 Composants UI (UI.tsx, Modales)           |
| ├── lib/                | # 🧠 Services & Logique                        |
| │   ├── api.ts          | # Appels vers Render (Endpoints API)          |
| │   ├── auth.ts         | # Gestion des Tokens JWT & Sessions           |
| │   ├── context.tsx     | # State Management (Données partagées)        |
| │   ├── gemini.ts       | # Intégration Google AI Studio                |
| │   └── storage.ts      | # Persistance locale (AsyncStorage)           |
| ├── assets/             | # 🎨 Design (Fonts, Splash, Icones)            |
| ├── constants/          | # 🎨 Theme.ts (Codes couleurs Noir/Or)         |
| ├── app.json            | # Config Expo (SDK 54, Plugins)               |
| └── package.json        | # Scripts de build et dépendances             |

## BACKEND (Node.js / Express)

|                          |                                     |
|--------------------------|-------------------------------------|
| backend/                 |                                     |
| ├── src/                 | # 📁 Code Source Serveur             |
| │   ├── routes/          | # Les tuyaux (Endpoints)            |
| │   │   ├── auth.ts      | # Inscription / Connexion           |
| │   │   ├── haccp.ts     | # Enregistrement des températures   |
| │   │   ├── invoices.ts  | # Gestion des factures scannées     |
| │   │   ├── scan.ts      | # Traitement OCR/IA                 |
| │   │   └── suppliers.ts | # Base de données fournisseurs      |
| │   ├── services/        | # Logic métier complexe             |
| │   │   ├── gemini.ts    | # Analyse des images par l'IA       |
| │   │   └── supabase.ts  | # Liaison Base de données           |
| │   ├── middleware/      | # Sécurité (Vérification Auth)      |
| │   └── server.js        | # Point d'entrée principal          |
| ├── supabase-schema.sql  | # Plan des tables SQL (PostgreSQL)  |
| └── .env                 | # Clés Secrètes (À NE PAS PARTAGER) |

---

## Écosystème Cloud & API

| Service             | Rôle                       | Statut                                                                                  |
|---------------------|----------------------------|-----------------------------------------------------------------------------------------|
| Render              | Hébergement du Backend     | <a href="https://chefgestion-pro.onrender.com">https://chefgestion-pro.onrender.com</a> |
| Supabase            | Base de données PostgreSQL | Cloud (Stockage sécurisé)                                                               |
| Google Cloud Vision | Extraction de texte (OCR)  | API Key active                                                                          |
| Google AI Studio    | Analyse intelligente (IA)  | Modèle : <b>Gemini 1.5 Pro</b>                                                          |
| Stockage Local      | AsyncStorage               | (2.2.0) pour garder ta session active même sans Wi-Fi.                                  |

## Procédure de Sauvegarde & Déploiement (Git)

Pour enregistrer ton travail sur **GitHub** et **mettre à jour automatiquement ton serveur sur Render**, utilise la méthode **"Add-Commit-Push"** depuis ton terminal :

### 1. Préparer les fichiers

On indique à Git quels fichiers "monter" pour la sauvegarde.

Terminal

```
git add .
```

### 2. Valider les modifications

On met une étiquette précise sur le travail effectué.

Terminal

```
git commit -m "Ajoute: Description de ta modification"
```

💡 **Note du Chef** : \* Vérifie toujours ton statut avec **git status** avant de pousser.

- **En rouge** : Les fichiers modifiés mais pas encore "préparés" (not stage).
- **En vert** : Les fichiers prêts à être inclus dans ton prochain commit.
- **Les fichiers non suivis** : Ceux que Git ne connaît pas encore.

*C'est l'outil parfait pour vérifier que tu n'es pas en train d'envoyer par erreur ton fichier **.env** qui contient tes clé API ou des fichiers temporaires.*

### 3. Envoyer sur le Cloud

On pousse les données vers le dépôt (**GitHub**), ce qui déclenche le **build sur Render**.

Terminal

```
git push origin main
```

## Build & Déploiement (APK / iOS)

Tes scripts **eas build** sont déjà configurés pour générer tes versions de test :

Si tu fais une petite modif de texte ou de couleur dans le **frontend** et que tu ne veux pas recréer un APK complet (ce qui est long), tu peux faire :

*Terminal*

```
eas update --branch preview
```

Cela envoie la mise à jour "par les airs" à ton APK déjà installé (au prochain redémarrage de l'app).

### Android (APK Preview)

Pour installer l'app en dur sur ton téléphone :

*Terminal*    **# Se placer dans le dossier frontend/** `cd frontend`

```
npm run build:android
```

Le lien de téléchargement est généré sur [Expo.dev](https://expo.dev), ou un QR Code s'affiche dans le terminal afin d'installer l'APK.

### iOS (Via Expo Go pour tiers)

Pour faire tester l'app à un proche sans compte Apple Developer payant :

1. Ajoute l'utilisateur sur **Expo.dev**.
2. Publie la version : eas update.
3. L'utilisateur ouvre le lien dans **Safari** sur son iPhone.
4. Cliquer sur **"Open with Expo Go"**.

---

## Outils & Services Utilisés

| Outil                      | Rôle                                                    |
|----------------------------|---------------------------------------------------------|
| <b>Expo 54</b>             | Framework principal de l'application.                   |
| <b>Gemini / Claude</b>     | Tes commis de cuisine pour le code et le debug.         |
| <b>Google Cloud Vision</b> | OCR pour "lire" tes étiquettes de produits.             |
| <b>Google AI Studio</b>    | Cerveau IA pour l'analyse des données scannées.         |
| <b>Supabase / Render</b>   | Ton infrastructure serveur et ta chambre froide (Data). |
| <b>GitHub</b>              | Ton carnet de recettes (Sauvegarde du code).            |

---

## ⚠ Limitations des API (Versions Gratuites)

- **Google Gemini API** : Limité à 20 requêtes / jour.
- **Google Cloud Vision API** : Limité à 33 requêtes / jour.
- **Render** : Mise en veille auto après 15 min. (Prévoir 30s de chargement au réveil).
- **Supabase** : 500 MB de stockage / 5 GB de bande passante mensuelle.

---

## 🔧 Dépannage (Troubleshooting)

**Erreur "Metro non trouvé" ou IP incorrecte** Si tu changes de réseau (ex: de chez toi à la Cabana), vérifie que ton adresse IP est toujours **192.168.1.122**. Sinon, mets-la à jour dans ton **package.json** sous le script **"start"**.

- **Nettoyage complet (Reset)**

**npm run start** # Le -c inclus videra déjà le cache Metro

- **Erreur Fonts (Cinzel / EB Garamond)** Si les polices luxueuses ne s'affichent pas :

**npx expo install @expo-google-fonts/cinzel @expo-google-fonts/eb-garamond @expo-google-fonts/dm-sans**

- **Sync Backend** : Si les températures ne s'enregistrent pas, vérifie que le serveur Render n'est pas en cours de maintenance, ou que les quotas n'ont pas été atteints.